الحلقات المتداخلة
(Nested Loops) في #C

خطوة بخطوة لفهم المنطق البرمجي 🚀

إعداد: الأستاذ محمود اغبارية

أولاً: عملية بحلقة واحدة (سطر من النجوم) 🌟

لنبدأ بكتابة عملية بسيطة مهمتها طباعة **سطر واحد** يتكون من عدد معين من النجوم.

```
static void PrintRow(int n) {
    for (int i = 1; i < = n; i++) {
        Console.Write("* ");
    }
    Console.WriteLine();
}
```

> هذه العملية تستقبل العدد **n** وتنفذ حلقة تكرارية عادية من أجل طباعة النجوم بجانب بعضها.

ثانياً: استدعاء العملية داخل حلقة (في Main) 🔄

الآن، ماذا لو أردنا طباعة هذا السطر **عدة مرات**؟ سنستدعي العملية داخل حلقة أخرى في Main.

```
static void Main() {
    for (int row = 1; row < = 3; row++) {
        PrintRow(5); // طباعة سطر من 5 نجوم
    }
}
```

> كررنا استدعاء العملية **ثلاث مرات**. النتيجة؟ ستتم طباعة 3 أسطر متتالية، كل سطر فيه 5 نجوم.

لحظة إدراك.. 💡

بما أننا وضعنا استدعاء لعملية تحتوي على **حلقة**، بداخل **حلقة** أخرى في الدالة الرئيسية...

فنحن في الواقع قد نفذنا حلقة بداخل حلقة!

وهذا بالضبط ما نسميه في البرمجة بالحلقات المتداخلة (Nested Loops).

ثالثاً: الكود كاملاً (الدمج) 🧩

بدلاً من كتابة عملية خارجية، يمكننا دمج الحلقتين معاً في مكان واحد:

```
// الحلقة الخارجية (للصفوف)
for (int row = 1; row < = 3; row++) {

    // الحلقة الداخلية (للأعمدة/أفقياً)
    for (int col = 1; col < = 5; col++) {
        Console.Write("* ");
    }

    // سطر جديد بعد انتهاء الحلقة الداخلية
    Console.WriteLine();
}
```

سؤال تفاعلي سريع 🧠

في الكود السابق:

- الحلقة الخارجية دارت **3 مرات** (3 صفوف).
- في كل دورة للحلقة الخارجية، دارت الحلقة الداخلية دورتها كاملة **5 مرات** (5 أعمدة).

كم إجمالي عدد النجوم المطبوعة؟

3 × 5 = 15 أمر طباعة!

قاعدة ذهبية: لمعرفة عدد مرات تنفيذ الأمر الموجود داخل الحلقة الداخلية، نضرب عدد دورات الحلقة الخارجية في عدد دورات الحلقة الداخلية.

كيف تعمل معاً؟ (تشبيه الساعة الرقمية ⏱️)

تخيلوا **الساعة الرقمية** التي نلبسها، كيف تعمل الدقائق والثواني؟

- **الساعات (الحلقة الخارجية):** تتحرك ببطء شديد.
- **الدقائق (الحلقة الوسطى):** تنتظر الثواني لتنهي دورتها كاملة لكي تزيد خطوة.
- **الثواني (الحلقة الداخلية):** هي الأسرع! تكمل دورتها بالكامل مع كل خطوة للخارجية.

تطبيق قاعدة الوقت برمجياً ⚙️

القاعدة الذهبية للحلقات المتداخلة:

- الحلقة الخارجية لا تزيد خطوة إلا بعد أن تنهي الحلقة الداخلية **دورتها كاااااملة**.
- الحلقة الداخلية سريعة وتعمل بجهد وتكرار أكبر.
- البرنامج "يُعلق" داخل الحلقة الداخلية حتى ينكسر شرطها، ثم يعود لخطوة جديدة في الخارجية.

تمرين تفكير وتتبع 📝

لقد قمنا برسم مستطيل بفضل ثبات عدد النجوم..

ماذا لو أردنا تغيير طول السطر في كل مرة لرسم أشكال هندسية مختلفة؟

التحدي: رسم مثلث من النجوم ▲

المطلوب كتابة كود يطبع الشكل التالي بالضبط:

```
*
* *
* * *
* * * *
* * * * *
```

السطر الأول فيه نجمة، والثاني نجمتان..

كيف نجعل توقف الحلقة الداخلية يعتمد على رقم الحلقة الخارجية بدلاً من رقم ثابت؟

الحل والشرح: كسر القاعدة الثابتة 💡

- نجعل شرط توقف الحلقة الداخلية (الأعمدة) يعتمد بشكل مباشر على المتغير الخاص بالحلقة الخارجية (الصفوف).
- بدلًا من الدوران حتى الرقم الثابت 5، نطلب الدوران حتى المتغير row.

```
for (int row = 1; row < = 5; row++) {
    // التوقف يعتمد الآن على رقم السطر الحالي!
    for (int col = 1; col < = row; col++) {
        Console.Write("* ");
    }
    Console.WriteLine();
}
```

جاهزون للمزيد من التحديات البرمجية؟ 🚀

الحلقات المتداخلة هي أداة قوية جداً.

تذكروا دائماً: ابدأوا بفهم الحلقة الداخلية كعملية منفصلة، ثم تخيلوها تتكرر بشكل كامل بواسطة الحلقة الخارجية!

هل من أسئلة إضافية حول التتبع؟ ✋